

The Five Workflows: Arrangements of Clerks, Paths Fixed in Code

Chapter 3, objective 1.3b. Chaining, routing, parallelization, orchestrator-workers and evaluator-optimizer. Exhibits E2, E3, E4 and E6 decoded.

A **workflow** is several augmented-LLM calls whose sequence your code decides before any model speaks. The model fills in steps. It never chooses them. Five named arrangements cover the exam, and the book says to learn each as a chain of four: shape, when, Kaveri scene, trap.

3.1 Prompt chaining (E2): the assembly line

The picture. Three clerks in a row. The first sorts the letter, the second drafts the reply, and between them stands a supervisor with a checklist who bins anything malformed. Nobody improvises the order. Accuracy goes up. Latency goes up too, because you queued three desks.

When: the task decomposes into fixed subtasks known in advance, and you will trade latency for accuracy. The programmatic gate between steps is the pattern's signature.

Lines	What E2 does	In plain words
1	VALID is a set of six category words.	The supervisor's laminated checklist. Six words, nothing else passes.
3 to 5	classify() asks a small model for one word, the category only.	Desk one. A small cheap model is enough to say customs. Each desk is exactly as smart as its step needs.
7 to 10	gate() raises ValueError if the category is not in VALID.	The gate is code, not model. If desk one mumbles something customs-ish, the line stops instead of desk two writing a confident reply to a nonsense category. Deterministic checks between steps are the whole reason chaining beats one big prompt.
12 to 15	draft_reply() asks the main model for a courteous reply under 120 words with no compensation promises.	Desk two writes to Padma. It is told the category and the boundaries. It never re-decides the category. The line has one direction.
17 to 18	reply = draft_reply(EMAIL, gate(classify(EMAIL)))	The whole line in one expression: classify, gate, draft. Your code owns this order forever. When a stem says the steps are the same every time, this, not an agent, is the answer.

Trap: a chain where a lookup would do. If the category is already a field on the ticket, desk one is an LLM re-deriving what a database column knows. Over-engineering wears a lab coat here: it looks diligent. Check the stem for data that already exists before you spend a model call creating it.

3.2 Routing (E3): the hospital triage nurse

The picture. One nurse at the door, thirty seconds per patient, sending each to the right specialist ward. The nurse treats nobody. A cheap decision at the front, expensive expertise only where it is needed.

When: distinct input categories exist, each better served by a different prompt or model, and classification is reliable. Routing separates concerns: improving the refund specialist cannot break the damage specialist.

Lines	What E3 does	In plain words
1 to 8	SPECIALISTS maps each category to a model and a system prompt: refund to the large model, customs to the mid model, delay to the small model.	Three door signs. Read the model column with Domain 2 eyes: the refund ward gets the big model because money and judgement live there; delays get the small one because an apology has a template's heart. Routing is where model selection stops being one decision and becomes a decision per category.
10 to 13	route() reuses E2's classify and gate, then looks up the specialist with .get(cat, default delay) and makes one call.	The nurse is E2's classifier, reused. One new idea: .get with a default. An unmapped category falls to a safe ward instead of crashing at two in the morning. Cheap insurance against a silent failure.
15	One email in, one specialist out.	The path was chosen by code reading one word. The model never saw a menu of wards. That is what keeps this a workflow, not an agent.

Trap: the router grading its own homework. A stem offering to let the specialist model also verify the routing was correct is inviting a circle: the expensive call re-doing the cheap call's job. If routing accuracy worries you, the answer lives in Domain 4, an evaluation set for the classifier, not in doubling the pipeline.

3.3 Parallelization (E4): three scouts, or three judges

The picture. Send three scouts down three different roads at once. The answer arrives when the slowest scout returns. That is sectioning. Ask three judges the same question and take the majority. That is voting. Same trick, running things at the same time, for opposite purposes: coverage versus confidence.

Lines	What E4 does	In plain words
1 to 5	check(carrier, cid) is an async function asking the small model for one ETA line.	One scout, one road. async means a scout who does not make the others wait at the gate while he walks.
7 to 10	sectioning() gathers check() for BlueDart, Delhivery and IndiaPost at once.	asyncio.gather releases all three together. Wall-clock time is the slowest scout, not the sum: three two-second calls take about two seconds instead of six. When a stem waves a latency budget at independent subtasks, this is the tell.
12 to 17	voting() asks the main model APPROVE or REJECT three times and takes the majority.	Same claim, three independent verdicts, majority rules. Cost triples for one decision, so voting is spent where being wrong is expensive, like refunds, not where it is cheap, like apologies.
19 to 20	Both halves run.	Notice what did not happen: no model chose to parallelize. Your code drew the fork. Workflow.

Trap: voting on facts. Voting raises confidence on judgement. It cannot vote a fact into existence. Three models guessing an ETA will agree on a hallucination as happily as on the truth. The scout for a fact is a tool call, not a ballot. Right mechanism, wrong axis: the reversed trade-off family.

Domain 3 connection: the carrier ETA call in One Ticket ran concurrently with the tail of retrieval and came back in 610 ms with a 3 second timeout. That was sectioning with a budget, the same shape as this exhibit.

3.4 Orchestrator-workers (E5): the site foreman

The picture. A foreman walks the site each morning and writes that day's task list himself. Nobody handed him one. Masons, electricians and plumbers each get a chit. The foreman reads their reports and synthesises. The list is different every morning because the site is.

When: subtasks cannot be predicted in advance, so the model must decompose. This is the boundary stone between workflows and agents. The *structure*, delegate then work then synthesise, is still fixed code. The *content* of the plan is the model's. E5's code and its line-by-line decode live in Part 5, where objective 1.5 needs them.

3.5 Evaluator-optimizer (E6): the writer and the editor

The picture. One clerk drafts. A second, colder clerk marks it against a rubric and hands it back. Two rounds of this and the letter is noticeably better. But without a page limit the pair would polish forever, so the editor's desk has a rule: two revisions, then it ships.

Lines	What E6 does	In plain words
1 to 2	draft_and_polish(email, max_rounds=2). First draft is free-form.	The letter to Padma gets an editor. Everything after this answers to the rubric.
3	for round_no in range(max_rounds)	The budget: at most two rounds. Law 3 in four characters. An unbounded while-loop here is the classic silent way to spend Rs 400 polishing a delay apology.
4 to 6	The evaluator gets the rubric (tone courteous? facts only? under 120 words?) and the draft. It replies PASS or one specific criticism.	The editor sees the rubric and the draft, nothing else. Clear criteria are the pattern's entry fee. No rubric, no evaluator, just two models exchanging vibes.
7 to 8	if verdict == PASS: break	The exit. PASS breaks early. Budget and exit: E7's agent loop will need exactly the same two organs, grown larger.
9 to 11	Revise with the criticism and the draft. Return the last draft.	Targeted feedback on a specific failure, which is precisely the iterative-refinement discipline the 50 percent row was testing. The letter that ships is the last draft, on PASS or when the budget runs out.

Trap: evaluator without criteria. Have a second model review it, with no rubric, is the sounds-diligent ghost in a new coat. It measures nothing. If the stem names no criteria, the evaluator option is decoration and the correct answer usually fixes the first prompt instead.

All five on one card

Pattern	Everyday picture	Signature in code	When	Trap
Chaining	assembly line with a supervisor	a gate function between steps	fixed steps, accuracy over latency	chaining what a lookup knows
Routing	triage nurse	a dict of specialists, one call	known categories, different handling	router grading its own homework
Parallel	three scouts or three judges	asyncio.gather	independent subtasks, or 3x cost for confidence	voting on facts
Orchestrator-workers	site foreman	model writes the plan, code steers	subtasks unpredictable	decomposing the known
Evaluator-optimizer	writer and editor	rubric, max_rounds, PASS	a rubric exists, drafts improve	evaluator without criteria

LAW 2 · Workflows when the path is known; agents when it isn't. All five arrangements share one property: code owns the path. The moment no path can be fixed in advance, and the model must look at each result to choose its next act, you have left this chapter.

***The one thing to remember.** Five arrangements of one clerk. Code picks the order every time. Learn each as shape, when, scene, trap, and you can recognise a diagram described in words.*